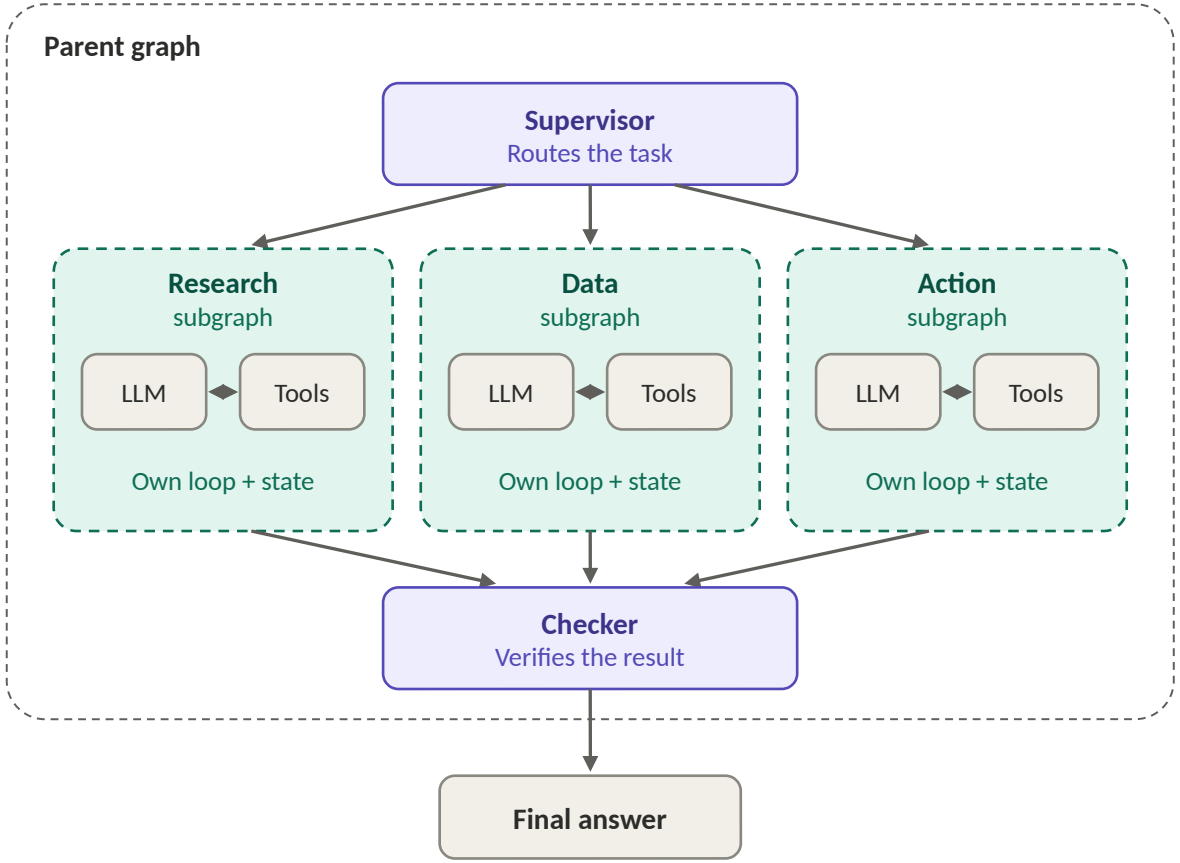
Multi-Agent Systems & Subgraph Architecture

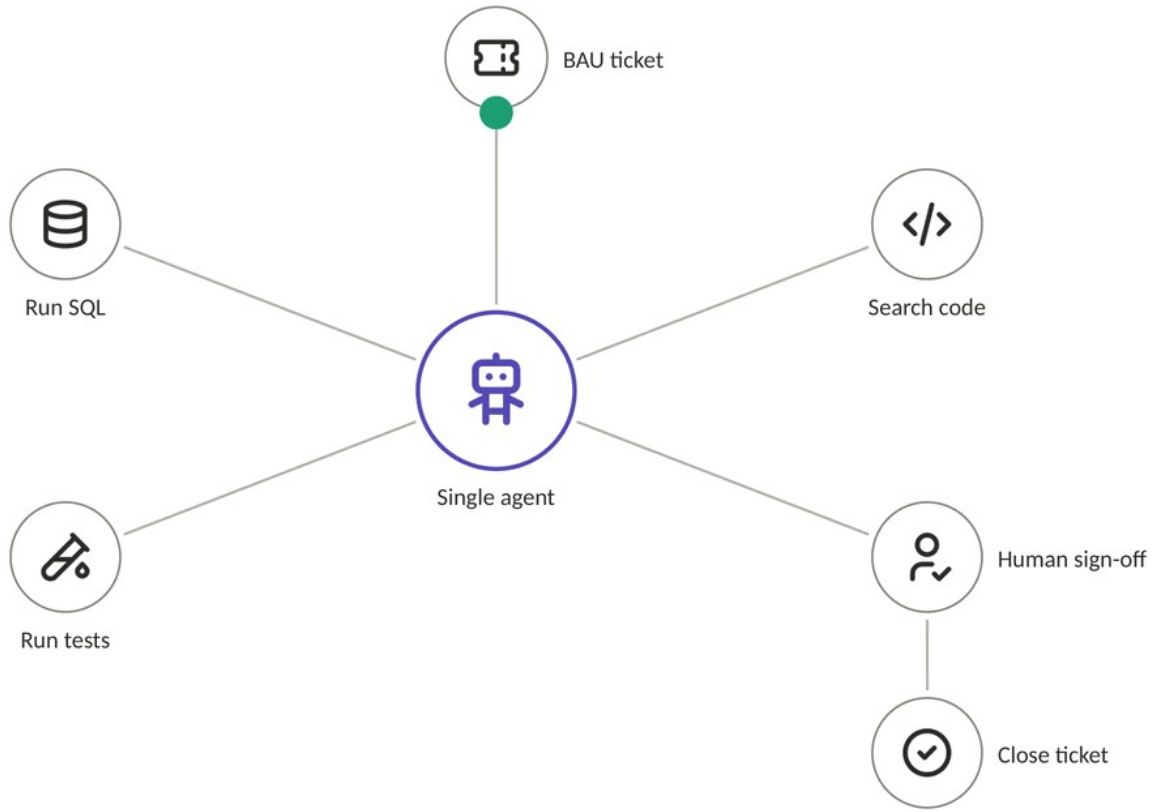
From one overloaded agent to a team of focused ones.

Every single-agent loop is steered by one field, **`finish_reason`**, and that's where the problems start.



Purple: control nodes · Dashed teal: subgraphs

Why Single Agents Fail · Happy Path



Happy flow · 1 ticket

Before we start

Same agent, same five tools. First, one ticket.

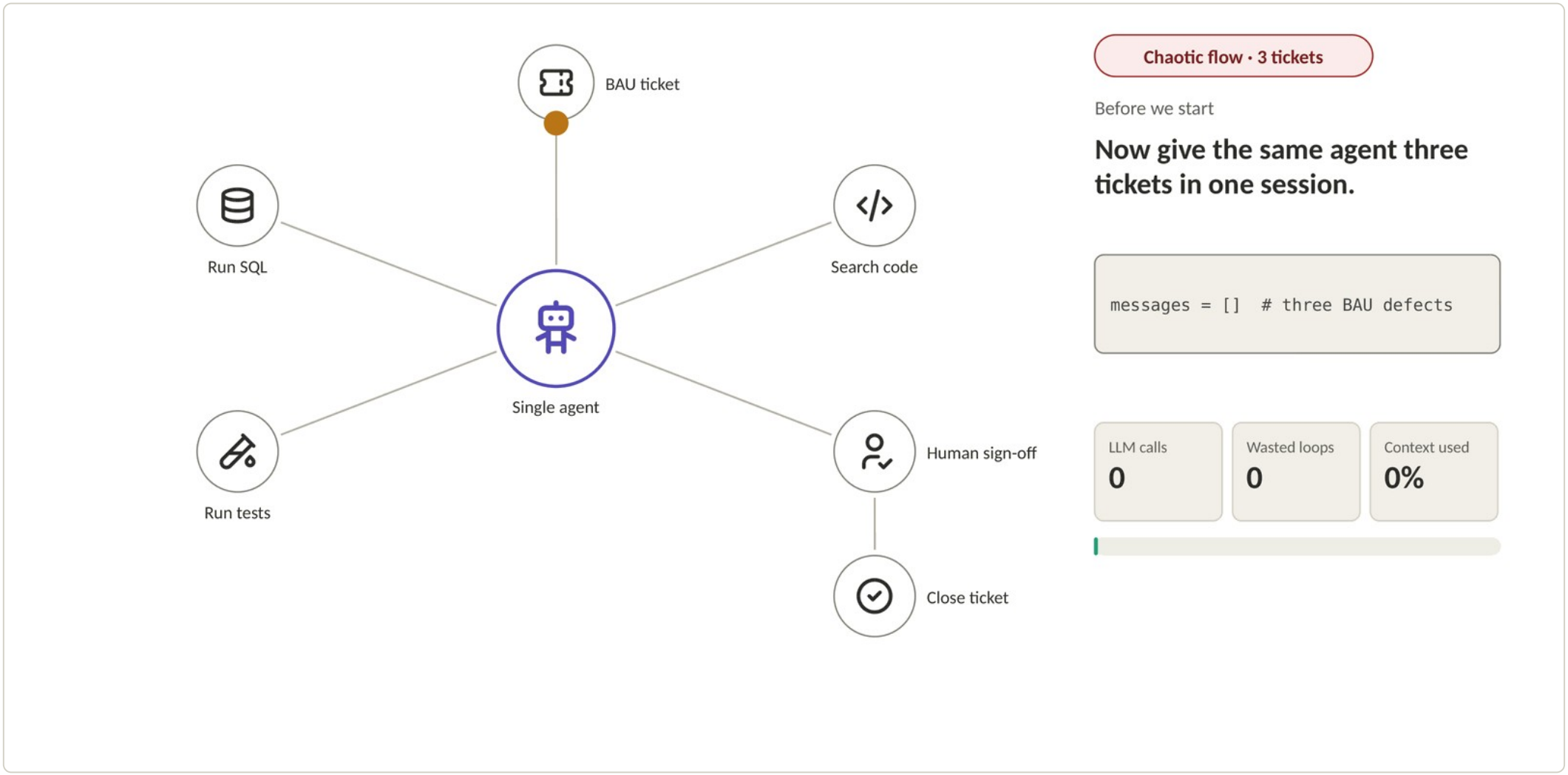
```
messages = []  # one BAU defect
```

LLM calls	Wasted loops	Context used
0	0	0%

▶ Play animation

Same agent, same five tools: one ticket, fixed and signed off in 4 LLM calls.

Why Single Agents Fail · Chaotic Path



▶ Play animation

A single agent is a monolith: it works for one task and fails like a monolith when you scale the work.

Thinking + Acting + Checking Are Mixed Together

One agent, one context window · time flows down ↓

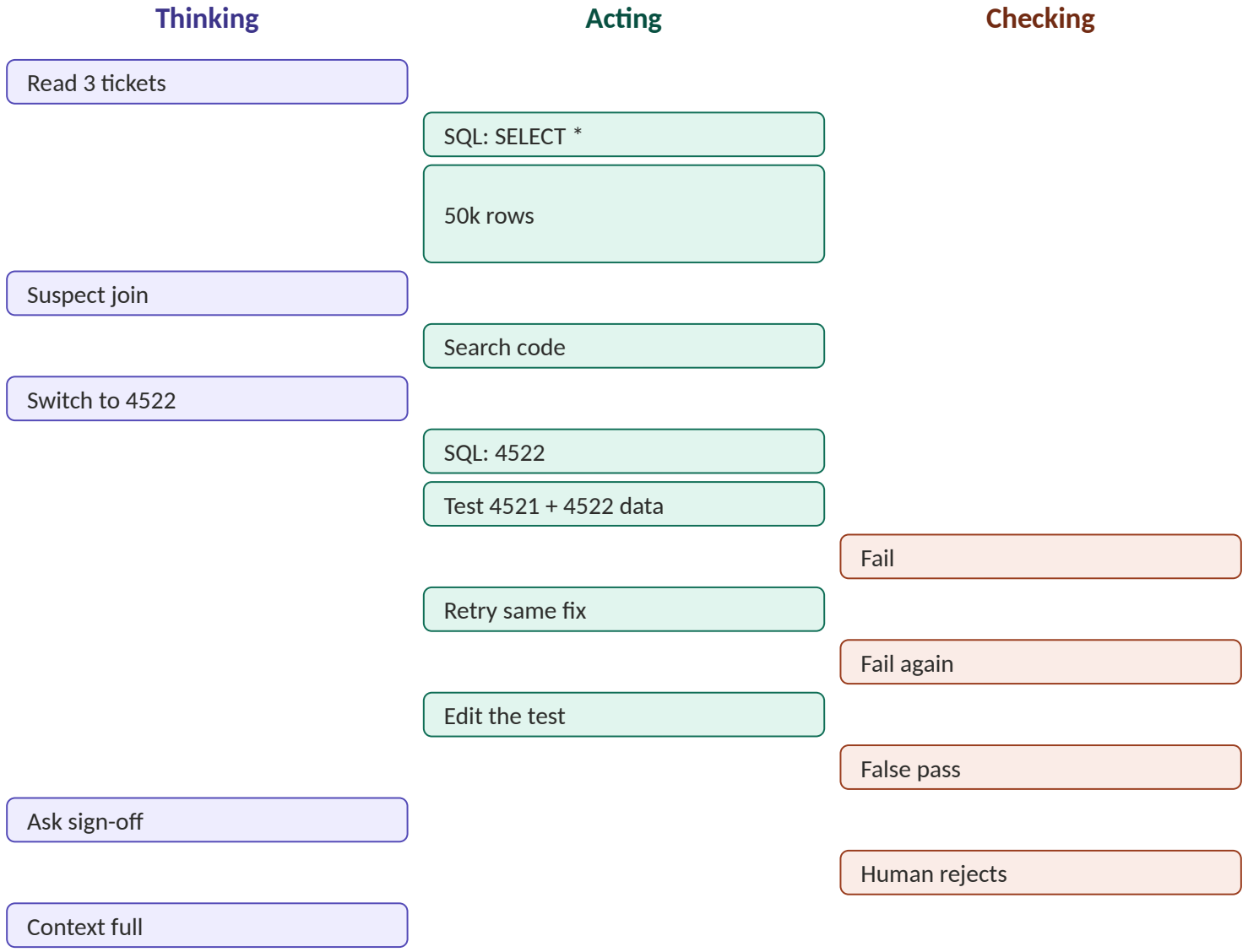
Read 3 tickets	`no_signal`
SQL: SELECT *	`tool_use`
50k rows	`tool`
Suspect join	`no_signal`
Search code	`tool_use`
Switch to 4522	`no_signal`
SQL: 4522	`tool_use`
Test 4521 + 4522 data	`tool_use`
Fail	`tool`
Retry same fix	`tool_use`
Fail again	`tool`
Edit the test	`tool_use`
False pass	`tool`
Ask sign-off	`end_turn`
Human rejects	`user`
Context full	`max_tokens`

Three jobs in one loop

- **Thinking:** which ticket, what cause, which tool next
- **Acting:** SQL, code search, tests
- **Checking:** did the fix work, does the evidence match

`finish_reason` only signals acting (tool_use) or stopping (end_turn). There is no "plan ready" and no "check passed".

Thinking + Acting + Checking Are Mixed Together



Three jobs in one loop

- **Thinking:** which ticket, what cause, which tool next
- **Acting:** SQL, code search, tests
- **Checking:** did the fix work, does the evidence match

`finish_reason` only signals acting (tool_use) or stopping (end_turn). There is no "plan ready" and no "check passed".

One loop, three jobs, one exit signal. We'd never let the maker check their own work. Why let an agent?

Failure Modes: What You Observe

Thinking failures

Goal drift

See: wanders off the original ask

Cause: long, crowded context

Example: asked to summarise a report, starts rewriting it

Wrong tool

See: right intent, wrong action

Cause: too many similar tools

Example: sends the email instead of drafting it

Acting failures

Infinite loop

See: same step again and again

Cause: no step limit, no memory of failures

Example: keeps re-searching the same flight

Error cascade

See: one bad step spoils the rest

Cause: no isolation or checkpoints

Example: wrong customer ID used in every later step

Checking failures

False success

See: says "done" when it isn't

Cause: grades its own work

Example: "report sent" but the attachment is empty

Premature stop

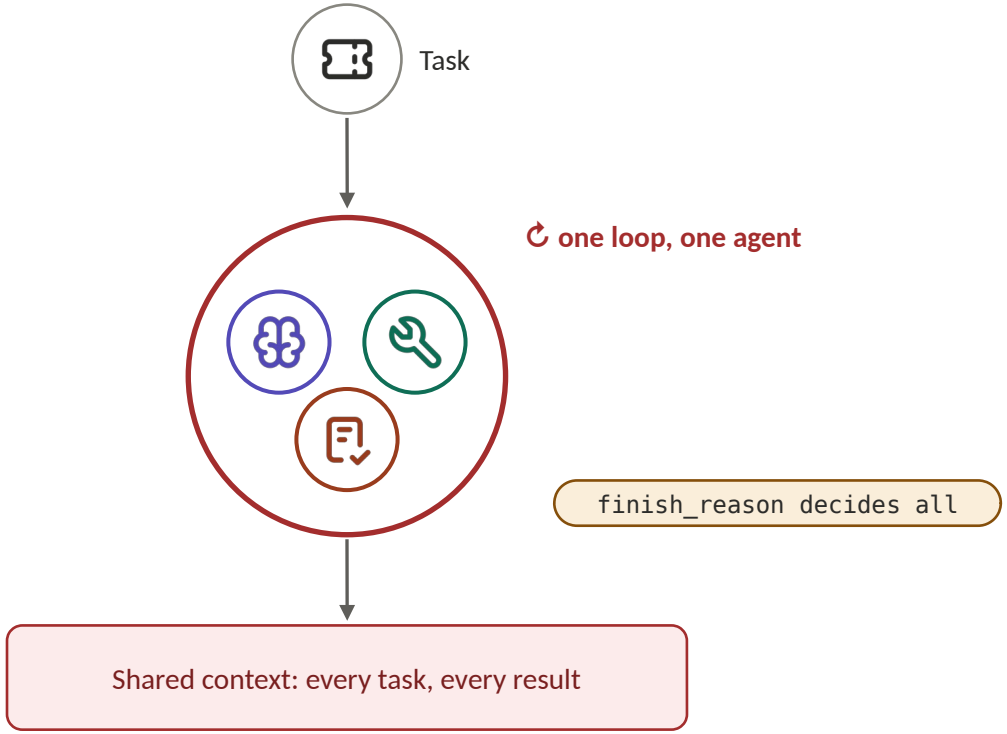
See: quits halfway, sounds confident

Cause: "stopped talking" read as "finished"

Example: answers 2 of 5 questions

Every failure is one of three jobs going wrong, and every cause is a design choice.

Root Cause: A System Design Problem



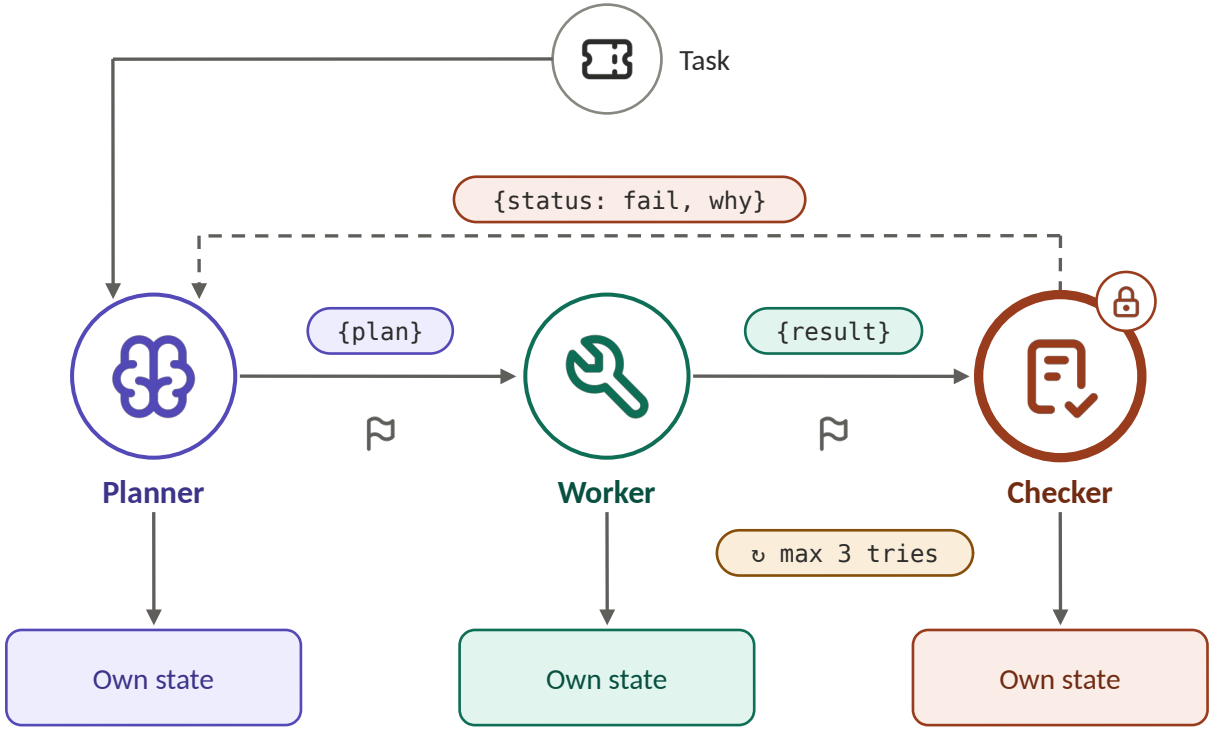
Before

Root cause

One agent does thinking, acting and checking, shares one context, and only stops when finish_reason says so.

- Separation of concerns
- Isolated state
- Bounded execution
- Maker-checker
- Explicit hand-offs

Root Cause: A System Design Problem



After · all five principles applied

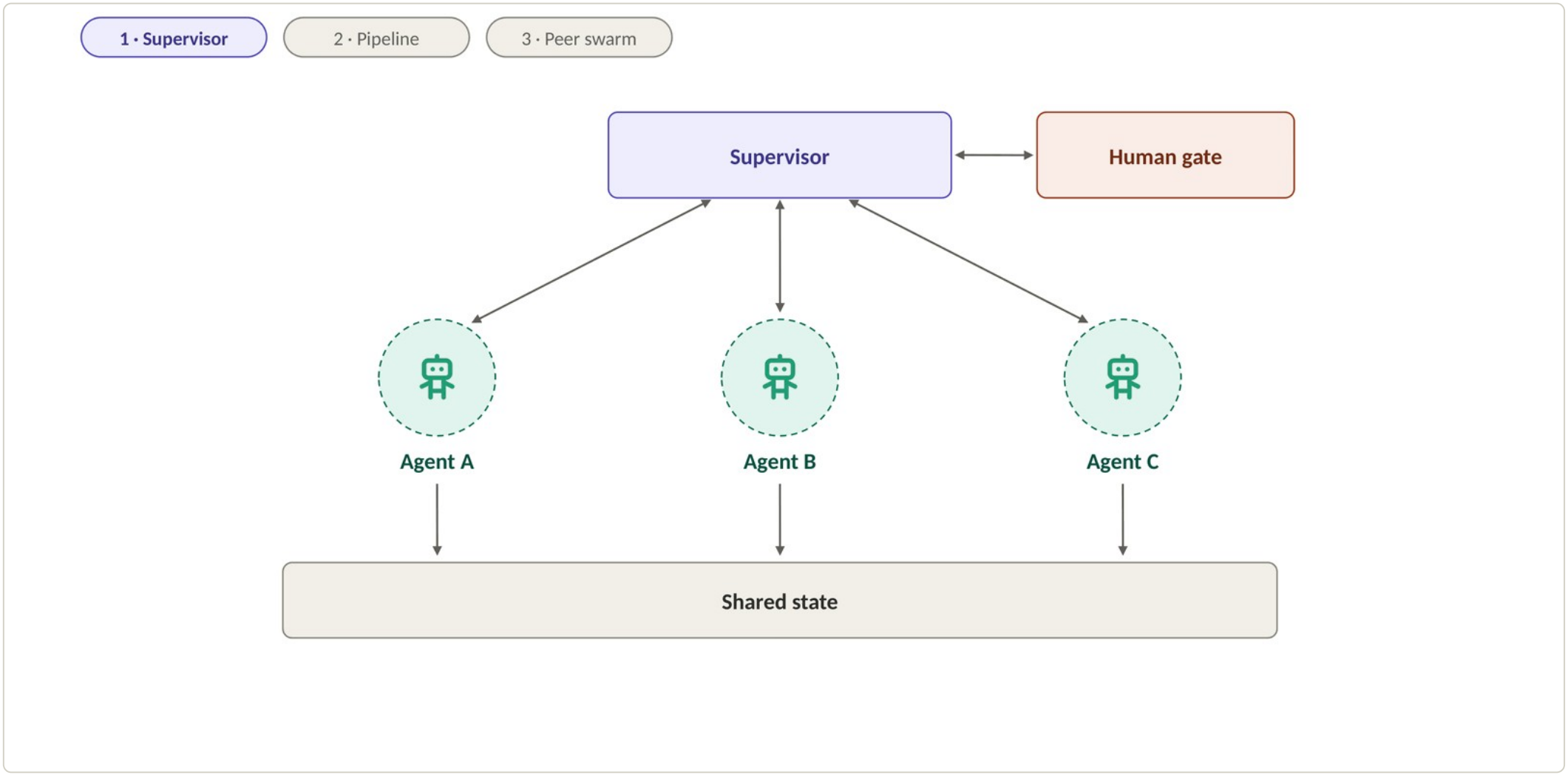
Explicit hand-offs

Each step returns a structured status instead of relying on finish_reason. This is a multi-agent system.

- **Separation of concerns**
- **Isolated state**
- **Bounded execution**
- **Maker-checker**
- **Explicit hand-offs**

This isn't an AI problem. It's a system design problem we've already solved, just not for agents yet.

The Core Idea of Multi-Agent Systems



▶ Play animation

A multi-agent system is a well-run team: one job each, clear hand-offs, a shared goal, and someone who signs off.